

Master's thesis

Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

August 13, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ohjaajat — Handledare — Supervisors

Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	August 13, 2026	21 pages, 1 appendix pages

Tiivistelmä — Referat — Abstract

Abstract to be written last.

ACM Computing Classification System (CCS)
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, FFI

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Software study track

Contents

1 Introduction 1

2 Background: Systems Languages for Latency-Critical Software 3

3 Problem Context: UE Context Management and Predictability 5

4 Objectives 7

5 Research Methodology 9

6 Design and Implementation 11

7 Experimental Setup and Evaluation 13

8 Discussion 15

9 Conclusion and Future Work 17

Bibliography 19

A Fault-injection example

1 Introduction

- The 5G RAN control plane (the gNB-CU-CP) now runs as software on ordinary servers. It is large, it keeps state for every connected device, it serves many requests at once, and it has only a few milliseconds to complete each procedure [1]. One defect, or one regression in the slowest requests, therefore degrades the whole site. The work is carried out at Nokia on a UE Context Management service modelled on a real gNB-CU-CP component.

- This software is written mostly in C++. C++ gives the control over memory and timing that such a system needs, but it leaves memory-safety and concurrency mistakes to the programmer [2]. Rust promises the same performance while rejecting those mistakes at compile time [3].

- The problem is a lack of evidence. Telecom vendors are under growing pressure to move to memory-safe languages [4], but the closest prior work does not isolate the language. Karlbäck and Orö compare Rust and Go against C++ for RAN applications and keep the runtime in play [5]; Arvidsson and Kaller find Rust async runtimes diverging from C++ only in the tail, but their comparison is between asynchronous frameworks [6]. No controlled study holds the specification, concurrency model, and build settings fixed while sweeping past saturation and measuring incremental migration cost.

- Four research questions frame the work. RQ1 compares latency predictability (mean, p99, p99.9, jitter) under increasing load and overload with the concurrency model held fixed [7]. RQ2 asks which defect classes Rust removes at compile time, at what stage the rest are detected, and which classes it does not prevent. RQ3 asks what overhead Rust's safety model imposes on throughput, CPU, and memory. RQ4 asks what incremental adoption costs in effort, FFI interoperability, and risk inside an existing C++ RAN codebase.

- The work contributes five things: a reproducible, telecom-relevant benchmarking framework; a frozen shared specification with two parity-tested implementations; a quantitative comparison that emphasises tail latency under overload; a defect-class taxonomy that marks where Rust's compile-time guarantee stops;

and evidence-based guidelines for incremental adoption, where the dominant cost is the learning curve rather than run-time [8].

2 Background: Systems Languages for Latency-Critical Software

- In the NG-RAN architecture the base station (gNB) is split into a central and a distributed unit, and the central unit is split again into a control plane and a user plane. UE Context Management sits in the control plane, the gNB-CU-CP, and reaches its neighbours over the standardised F1, E1, and NG interfaces [9].
- Deadlines are met or missed because of predictability, not average speed. The metrics that matter are therefore the p99 and p99.9 latencies and the jitter around them [7]. They have to be measured past the saturation point and recorded without coordinated omission, that is, without letting a load generator that has fallen behind hide its own slowest requests [10].
- C++ offers abstractions that cost nothing at run time, deterministic cleanup through RAII, and no garbage collector. Safety, however, remains the programmer's responsibility [2], and sanitizers report only the errors that a given test run happens to reach.
- Rust has the same run-time model and also no garbage collector, but it checks memory safety and freedom from data races at compile time, using ownership, borrowing, and the `Send`/`Sync` traits [11]. Code that needs to step outside those rules must be marked `unsafe`, which keeps such code small and easy to find.
- The comparison is therefore C++ against Rust. Garbage-collected languages are left out because their pauses are incompatible with the latency targets. The concurrency model is held fixed in both implementations, and the C ABI is treated as the realistic adoption path. No earlier study isolates the language this way [5].

3 Problem Context: UE Context Management and Predictability

- UE Context Management owns the lifecycle of one context per connected device. Each context moves through four states, Idle, Connected, Reconfiguring, and Released, driven by a small set of lifecycle operations. An operation that is illegal in the current state returns a typed error instead of being carried out.
- The subsystem was chosen because it is both representative and small enough to keep under control. It has mutable state per device, a strict lifecycle, heavy concurrency, and hard latency targets [1], and it is still simple enough to specify precisely and to test for identical behaviour in two languages.
- Five defect classes fall inside Rust's guarantee [11], and each has a natural form in this subsystem: use-after-free, double-free, iterator invalidation, data race, and unguarded shared mutation.
- Two further classes are included on purpose from outside that guarantee, a lock-order deadlock and a lost update. They keep the study from confirming a foregone conclusion and show instead where the guarantee stops.
- Scale is what makes the tail matter. One procedure triggers many operations and one site serves many devices, so a p99.9 measured on a single operation becomes a steady rate of degraded procedures in the field [7]. This is why the load sweep is pushed past the point where the service starts to fall behind.

4 Objectives

- A documented literature search found no study that holds a stateful control-plane subsystem fixed while measuring tail latency of Rust and C++ past saturation under the same concurrency model [5, 6].
- Five objectives follow from the research questions: measure predictability (O1), establish which defect classes are prevented (O2), measure the resource overhead (O3), build and measure a migration prototype (O4), and publish reusable artifacts and guidance (O5).
- Three hypotheses are stated in advance, so that the measurements can refute them. H1: below saturation the two implementations perform the same [12]. H2: Rust prevents at compile time the defect classes for which C++ can only offer best-effort detection. H3: the cost of calling Rust from C++ is negligible.
- The work succeeds if the evidence is good enough to base a decision on: repeated runs reported with confidence intervals [13], defect claims shown on programs that really compile and run, migration cost read off a working prototype, and every number reproducible from the published artifacts.
- The scope is drawn narrowly to protect internal validity. The full gNB, the PHY and L2 layers, the user plane, captured traffic traces, managed languages, and production operations are all left out.

5 Research Methodology

- The approach is empirical and constructive. Instead of comparing two systems that already exist and differ in many ways, the study builds a matched pair of implementations from one frozen specification, so that the language is the only systematic difference between them. Both use an identical thread-per-shard concurrency model, because that structuring itself changes tail latency [14].
- RQ1 and RQ3 are answered with an open-loop load sweep that runs from light load into overload on pinned, frequency-locked bare-metal hardware; open-loop means the generator keeps to its send schedule instead of waiting for replies. Latency is recorded in identically configured HDR histograms [10], and resource use is sampled through the operating system. Until that host is available, parity testing and preliminary benchmarks run on a WSL2 development machine, which is adequate for structure and median comparisons but not for tail claims.
- RQ2 is answered by controlled fault injection [15]. A fixed catalogue of defects is written in both languages, and the expected outcome of each one is recorded in advance, so that a surprise shows up as a discrepancy instead of being explained away afterwards.
- RQ4 is answered by a working prototype in which C++ drives the Rust core through a foreign-function interface. It yields three measurements: how large the seam is, how much `unsafe` code it needs, and what one call across it costs [16].
- Fairness is the central experimental control: the specification is frozen, allocator and optimisation settings are matched, and every point where the two languages had to differ is written down in a forced-idiom log. Results are reported as medians with bootstrap confidence intervals, together with exact paired permutation tests corrected for multiple comparisons by the Holm procedure [13]. All data is synthetic, so no data-protection concerns arise.

6 Design and Implementation

7 Experimental Setup and Evaluation

8 Discussion

9 Conclusion and Future Work

Bibliography

[1] 3GPP. *Service requirements for the 5G system; Stage 1*. Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip.

[2] L. Szekeres, M. Payer, T. Wei, and D. Song. "SoK: Eternal War in Memory". In: *Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P)*. 2013, pp. 48–62. DOI: 10.1109/SP.2013.13. URL: https://doi.org/10.1109/SP.2013.13.

[3] S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6.

[4] Cybersecurity and Infrastructure Security Agency and National Security Agency and Federal Bureau of Investigation and others. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously*. Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/case-memory-safe-roadmaps.

[5] R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307.

[6] A. Arvidsson and M. Kaller. "Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746.

[7] J. Dean and L. A. Barroso. “The Tail at Scale”. In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[8] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. “Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study”. In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[9] 3GPP. *NG-RAN; Architecture description*. Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip.

[10] G. Tene. *HdrHistogram: A High Dynamic Range Histogram, and the Coordinated Omission Problem*. Accessed August 2026. URL: https://hdrhistogram.org/.

[11] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. “Safe Systems Programming in Rust”. In: *Communications of the ACM* 64.4 (2021), pp. 144–152. DOI: 10.1145/3418295. URL: https://doi.org/10.1145/3418295.

[12] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. “The Case for Writing Network Drivers in High-Level Programming Languages”. In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892.

[13] A. Georges, D. Buytaert, and L. Eeckhout. “Statistically Rigorous Java Performance Evaluation”. In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033.

[14] P. Enberg, A. Rao, and S. Tarkoma. “The Impact of Thread-per-Core Architecture on Application Tail Latency”. In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874.

[15] R. Natella, D. Cotroneo, and H. S. Madeira. "Assessing Dependability with Software Fault Injection: A Survey". In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425.

[16] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33.

Appendix A Fault-injection example

- This appendix shows one case from the RQ2 catalogue, the use-after-free, in both languages side by side.
- In the C++ version a reference into the context map outlives the `erase` that frees the entry, and the program still compiles cleanly.
- The error appears only at run time, and only if AddressSanitizer is enabled on a path that a test actually exercises.
- The equivalent safe-Rust program does not compile at all: the borrow checker refuses to let the map be modified while a reference into it is still live (`E0499`) [3].
- Side by side, the two programs make the difference in kind of guarantee concrete instead of arguing it from language semantics.